

Ain Shams University

Faculty of Engineering

CSE331 – Data Structures and Algorithms

Colored Water Sort Solver Project

Student: Mahmoud Abdelaziz Mohammed

ID: 24p0485

Project Report

1. Project Overview

The project is a console-based C++ solver for the Colored Water Sort puzzle. The program receives the initial tube arrangement and searches for a valid solution with the minimum number of moves.

2. Student Responsibilities

Mahmoud Abdelaziz Mohammed — implementation of the solver, data-structure design, testing, debugging, documentation, GitHub repository preparation, and project report.

3. Data Structures

`vector<vector<int>>` represents the puzzle, with each inner vector representing one tube. `State` stores a puzzle arrangement together with its parent state index and the source/destination move. `queue<int>` supports BFS. A vector of visited states prevents repeated processing.

4. Algorithm

The solver uses Breadth-First Search. The initial arrangement is inserted into the queue. For each state, every source/destination pair is checked, valid next states are generated, visited states are skipped, and new states are queued. Since every valid pour is one move, BFS explores states in increasing move count, so the first solved state found is a minimum-move solution.

5. Code Structure

`getTopColor()` finds the top non-empty color. `getTopCount()` counts the connected top layers. `getEmptyPosition()` finds available space. `canPour()` validates a move. `pour()` applies a move. `isSolved()` checks the goal state. `sameState()` and `isVisited()` prevent duplicate exploration. `solve()` performs BFS and reconstructs the solution. `printState()` displays the final arrangement.

6. Complexity Analysis

Let S be the number of reachable states, N the number of tubes, and C the tube capacity. Each state considers $O(N^2)$ source/destination pairs. The simple vector-based visited search can require $O(S \cdot N \cdot C)$ work in the worst case. Memory is $O(S \cdot N \cdot C)$ for stored states and visited arrangements, plus the BFS queue. Practical performance depends strongly on the number of reachable puzzle states.

7. GitHub Repository

The GitHub repository contains the C++ source file, sample inputs, README, test results, and project documentation.

8. Test Cases

Test 1 — Official Example A: Solution found in 3 moves. Moves: Tube 1 → Tube 3, Tube 2 → Tube 1, Tube 2 → Tube 3. Final state: [1,1], [0,0], [2,2], [0,0].

Test 2 — Official Example B: No solution exists.

Test 3 — Additional test: Solution found in 1 move. Move: Tube 1 → Tube 2. Final state: [0,0], [1,1], [2,2], [0,0].

9. Conclusion

The final implementation provides a console-based Colored Water Sort solver using vector-based state representation, BFS, visited-state checking, and parent tracking. It solves the official solvable example in three moves, reports no solution for the official unsolvable example, and passes an additional test case.